

# Deployment Guide

---

MarketFlow has two deployable parts:

- **server/** — FastAPI app, deployed to **Render** (Docker web service) with a Postgres database and Redis cache.
- **client/** — React + Vite app, deployed to **Vercel** (static SPA).

The current live server is hardcoded in **client/src/lib/utils.ts** (**SERVER\_HOST**), so any change to the server's URL must be reflected there before redeploying the client.

---

## 1. Prerequisites

- A [Render](#) account (for the API, Postgres, and Redis).
  - A [Vercel](#) account (for the client).
  - Repo pushed to GitHub (both Render and Vercel deploy from a connected Git repo).
- 

## 2. Deploy the database and cache (Render)

1. In the Render dashboard, click **New** → **PostgreSQL**.
    - Name it (e.g. **marketflow-db**), pick a region close to where the API will run.
    - Once created, copy the **Internal Database URL** — you'll use it as **DB\_URL**.
  2. Click **New** → **Key Value** (Render's managed Redis).
    - Name it (e.g. **marketflow-cache**).
    - Note the **Host** and **Port** from its connection info — used as **CACHE\_HOST** / **CACHE\_PORT**.
- 

## 3. Deploy the API (Render Web Service)

1. Click **New** → **Web Service** and connect the GitHub repo.
2. Set **Root Directory** to **server**.
3. Render will detect **server/Dockerfile** automatically — choose **Docker** as the environment.
4. Set the instance type/region, then configure **Environment Variables** (do not use **.env.dev.sample** values in production):

| Key                   | Value                                    |
|-----------------------|------------------------------------------|
| <b>ADMIN_USERNAME</b> | your chosen admin username               |
| <b>ADMIN_PASSWORD</b> | a strong admin password                  |
| <b>SECRET</b>         | a long random string (used to sign JWTs) |
| <b>DB_URL</b>         | Internal Database URL from step 2        |

---

| Key        | Value                  |
|------------|------------------------|
| CACHE_HOST | Redis host from step 2 |
| CACHE_PORT | Redis port from step 2 |

5. Deploy. Render will build the Docker image and start it with:

```
fastapi dev ./main.py --host 0.0.0.0 --port 8000
```

Render auto-detects the port from `EXPOSE 8000` in the Dockerfile.

6. Once live, note the public URL Render assigns, e.g. `https://<your-service>.onrender.com`.

**Note:** the app calls `SQLModel.metadata.create_all(engine)` on startup (`server/main.py`), so tables are created automatically on first boot — no separate migration step is needed.

---

## 4. Point the client at the API

Edit `client/src/lib/utils.ts`:

```
export const SERVER_HOST = "<your-service>.onrender.com"
```

Commit this change before deploying the client.

---

## 5. Deploy the client (Vercel)

1. In Vercel, click **Add New** → **Project** and import the same GitHub repo.
2. Set **Root Directory** to `client`.
3. Vercel auto-detects Vite; confirm these build settings:
  - **Build Command:** `npm run build`
  - **Output Directory:** `dist`
  - **Install Command:** `npm install`
4. `client/vercel.json` already rewrites all routes to `index.html`, which is required for client-side routing (`react-router`) to work on refresh/deep links — no extra config needed.
5. Deploy. Vercel gives you a `*.vercel.app` URL (add a custom domain under Project Settings if desired).

---

## 6. Verify

1. Open the deployed client URL.
2. Confirm it can reach the API (check the Network tab for successful calls to `SERVER_HOST`).
3. Test login/signup and a stock action to confirm the DB and Redis cache are wired correctly.

4. On Render, check the web service logs for errors (`DB_URL/CACHE_HOST` typos are the most common issue).

---

## 7. Redeploying after changes

- **Server:** push to the connected branch — Render auto-deploys (or trigger manually from the dashboard).
- **Client:** push to the connected branch — Vercel auto-deploys. Remember to update `SERVER_HOST` first if the API URL changed.

---

## Local development (for reference)

`compose.yaml` at the repo root spins up all four services together for local dev:

```
docker compose up
```

This runs the client (`:3000`), server (`:8000`), Redis (`:6379`), and Postgres (`:5432`) using `server/.env.dev.sample` as env vars — it is **not** used in production deployment, but is useful to sanity-check the app before deploying.